

CSE422 Lab Project Report

Shafe Al Toha (22301296)

05-05-2026

Course: Artificial Intelligence (CSE422)

Section: 5

Semester: Spring 2026

Project Title:

E-commerce Shipping Dataset Analysis

Dataset Used:

E-commerce Shipping Dataset.csv

**Department of Computer Science and Engineering
BRAC University**

Contents

1	Introduction	1
2	Dataset Description	1
3	Dataset Pre-processing	2
3.1	Identified Faults in Dataset	2
3.2	Solutions Applied	2
3.3	Final Dataset After Preprocessing	3
3.4	Summary of Preprocessing Steps	3
4	Dataset Splitting	3
4.1	Splitting Strategy	3
4.1.1	Train-Test Split	3
4.1.2	Validation Split	3
5	Model Training & Testing	3
5.1	K-Nearest Neighbors (KNN)	4
5.2	Decision Tree	4
5.3	Logistic Regression	4
5.4	Neural Network	4
5.5	Linear Regression (Discussion)	4
5.6	Unsupervised Learning: K-Means Clustering	4
5.6.1	Dataset Preparation for Clustering	5
5.6.2	Choosing Optimal K (Elbow Method)	5
5.6.3	Clustering Performance	5
5.6.4	Final Analysis	6
6	Model Selection & Comparison Analysis	6
6.1	Accuracy Comparison	7
6.2	Precision and Recall Comparison	8
6.3	Confusion Matrix Analysis	8
6.4	ROC Curve Analysis	11
7	Conclusion	11

1 Introduction

As e-commerce grows rapidly, getting products delivered on time has become most important for both companies and customers. In this project, we're building a machine learning model to predict whether a shipment will arrive on time or get delayed. Basically a simple yes/no classification(binary) problem. The dataset that we're working with has information about shipments, customer interactions, product details, and logistics. We tried a few different approaches including K-Nearest Neighbors, Decision Tree, Logistic Regression, and Neural Networks to see which one works best. We also used K-Means clustering to spot any hidden patterns in the data. At the end of the day, this project shows how machine learning can actually help to solve real logistics problems.

2 Dataset Description

The dataset contains 10,999 shipment records with 11 features after dropping the ID column. Here we are dealing with a classification problem since the target variable is "Reached.on.Time_Y.N" that just tells us whether a shipment arrived on time (1) or not (0), therefore it is a simple binary classification.

Out of the 11 features, 7 are numerical such as customer care calls, ratings, product cost, prior purchases, discount offered, weight in grams, and the target variable itself. The remaining 4 features are categorical those are warehouse block, shipment mode, product importance, and gender. Since machine learning models can't work with text directly, we had to encode these categories. We have used one-hot encoding for warehouse block, shipment mode, and gender, while product importance got ordinal encoding since low ; medium ; high makes logical sense, there is no missing value anywhere in the dataset, so we didn't have to fix this.

But, the dataset is a bit imbalanced. About 60% of shipments (6,563 records) arrived on time, while the remaining 40% (4,436 records) got delayed. It's not severely imbalanced though, just moderately skewed, so we could work with it without doing anything too drastic, it will create misleading on prediction. Figure 1 shows this distribution clearly.

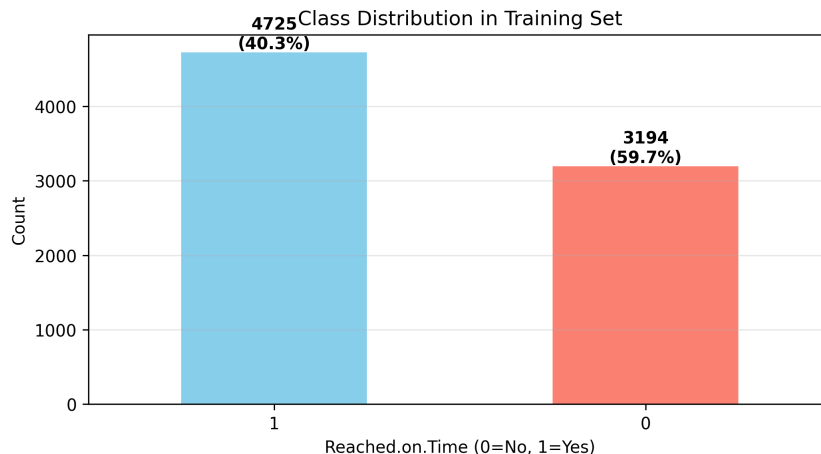


Figure 1: Class Distribution of Shipment Delivery

The correlation heatmap showed some interesting patterns. Discount offered seems to have a noticeable impact on delivery outcomes, and weight also plays a role. But honestly,

most features don't correlate strongly with the target, which is actually fine since it means no weird multicollinearity between features. Figure 2 presents the correlation matrix of all features.

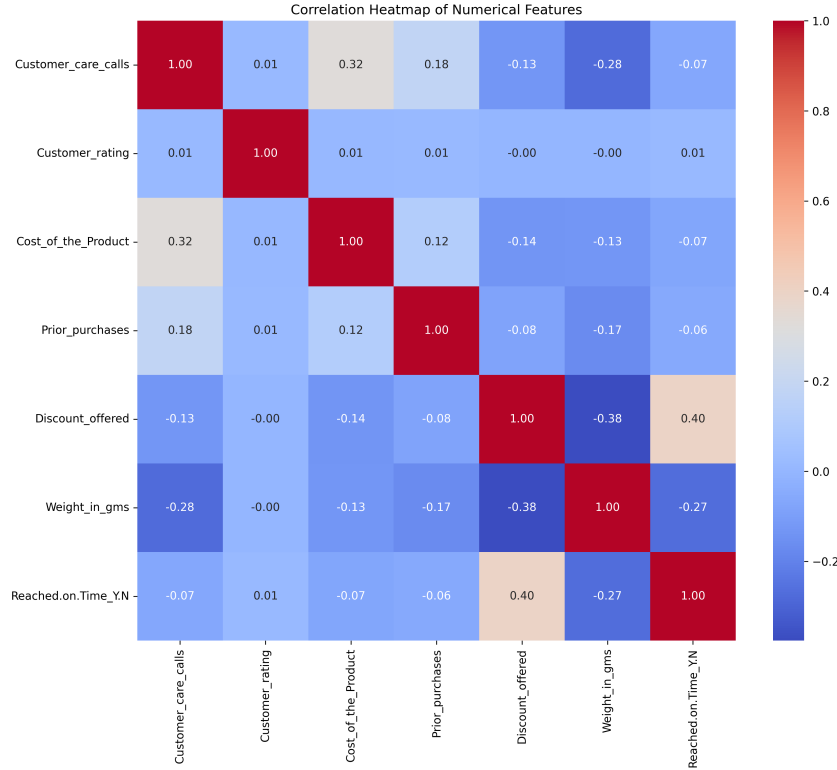


Figure 2: Correlation Heatmap of Features

3 Dataset Pre-processing

Data preprocessing ensures the dataset is clean, consistent, and suitable for model training. Techniques applied include handling missing values, encoding categorical variables, and feature scaling.

3.1 Identified Faults in Dataset

First, no missing values were found in any column, confirming complete data. Second, the dataset contains several categorical features including warehouse block, mode of shipment, product importance, and gender, which machine learning models cannot directly process. Third, feature scaling was required since numerical features have significantly different ranges. For instance, weight ranges from 1000 to 7800, discount ranges from 1 to 65, while customer rating ranges only from 1 to 5. Features with larger ranges may dominate smaller ones, leading to biased performance.

3.2 Solutions Applied

No missing value handling was required as the dataset was already clean. For categorical variables, product importance was ordinally encoded (low=0, medium=1, high=2) to preserve its inherent order. The remaining categorical features were one-hot encoded, which

prevents models from assuming false relationships and avoids multicollinearity. For feature scaling, standardization was applied, making each feature's mean approximately zero and standard deviation one. This ensures equal feature contribution, improves distance-based models like KNN, and speeds convergence.

3.3 Final Dataset After Preprocessing

The final dataset has 10,999 rows and 15 columns, consisting of 14 input features and one output feature. All features are in numerical float64 format, while the target remains int64.

3.4 Summary of Preprocessing Steps

No missing values required handling. All categorical features were properly encoded. Standardization was applied for feature scaling. The dataset is now fully numerical and ready for machine learning models.

4 Dataset Splitting

To evaluate the performance of the machine learning models effectively, the dataset was divided into three subsets: training, validation, and test sets. This ensures that the model is trained, tuned, and evaluated on separate data, preventing overfitting and improving generalization.

4.1 Splitting Strategy

A stratified sampling approach was used to maintain the original class distribution of the target variable (Reached.on.Time_Y.N) across all subsets.

4.1.1 Train-Test Split

The dataset was first split into a training set containing 80% of the data and a test set containing the remaining 20%. Stratification was applied to preserve the ratio of classes, ensuring that both on-time and delayed deliveries were proportionally represented in each subset.

4.1.2 Validation Split

From the training set, a validation set was created. The validation set consisted of 10% of the training data, while the final training set comprised the remaining 90%. This validation set was used for hyperparameter tuning and model selection.

5 Model Training & Testing

Multiple supervised learning models were applied to solve the classification problem of predicting whether a shipment will reach on time. Additionally, the dataset was treated as an unsupervised problem using K-Means clustering to explore hidden patterns.

5.1 K-Nearest Neighbors (KNN)

KNN classifies data points based on the majority class of their nearest neighbors.

Best Hyperparameters: `n_neighbors = 5`, `weights = uniform`, `metric = euclidean`

Performance: Accuracy of 62.73% and Macro F1-score of 0.62

Analysis: The model performs moderately, showing better performance at predicting Class 1 (on-time delivery) than Class 0. It is sensitive to feature space and class overlap.

5.2 Decision Tree

Decision Tree splits the dataset into subsets based on feature conditions.

Best Hyperparameters: `max_depth = 5`, `criterion = gini`, `min_samples_leaf = 4`, `min_samples_split = 2`

Performance: Accuracy of 67.82% and Macro F1-score of 0.67

Analysis: This achieved the highest performance among all tested models. It shows very high recall for Class 0 (0.97) but lower recall for Class 1 (0.48), indicating imbalance in predictions.

5.3 Logistic Regression

A linear model used for binary classification.

Best Hyperparameters: `C = 100`, `solver = liblinear`

Performance: Accuracy of 63.77% and Macro F1-score of 0.63

Analysis: The model shows balanced performance across both classes. It works effectively after feature scaling and serves as a strong baseline model.

5.4 Neural Network

A Deep Neural Network was implemented using TensorFlow/Keras.

Architecture: Dense (128, ReLU), Dropout (0.3), Dense (64, ReLU), Dropout (0.3), Output layer (Softmax)

Training Details: 20 epochs, batch size of 32, Adam optimizer

Performance: Accuracy of 65.77% and Macro F1-score of 0.6575

Analysis: The model performs better than KNN and Logistic Regression. It learns non-linear patterns effectively and shows balanced performance across classes.

5.5 Linear Regression (Discussion)

Linear Regression is not suitable for this problem because the target variable is categorical (0 or 1) and it produces continuous outputs. Therefore, it was not used for final evaluation, and Logistic Regression is more appropriate.

5.6 Unsupervised Learning: K-Means Clustering

To explore hidden structures, K-Means clustering was applied after removing the target variable.

5.6.1 Dataset Preparation for Clustering

The fully preprocessed dataset was used with 14 features and 10,999 samples. The data was already scaled and encoded.

5.6.2 Choosing Optimal K (Elbow Method)

The Elbow Method was used to determine the optimal number of clusters (Figure 3).

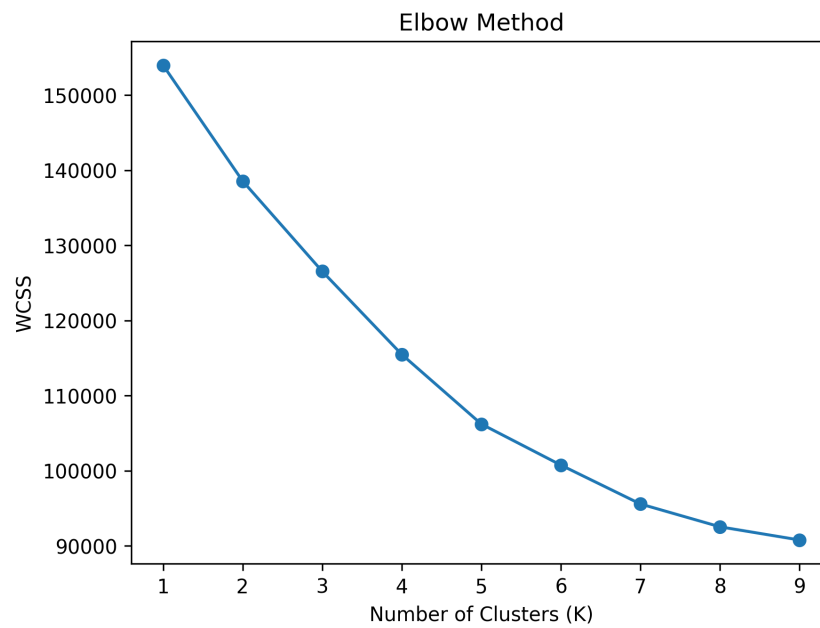


Figure 3: Elbow Method for Optimal K

5.6.3 Clustering Performance

(Figure 4)

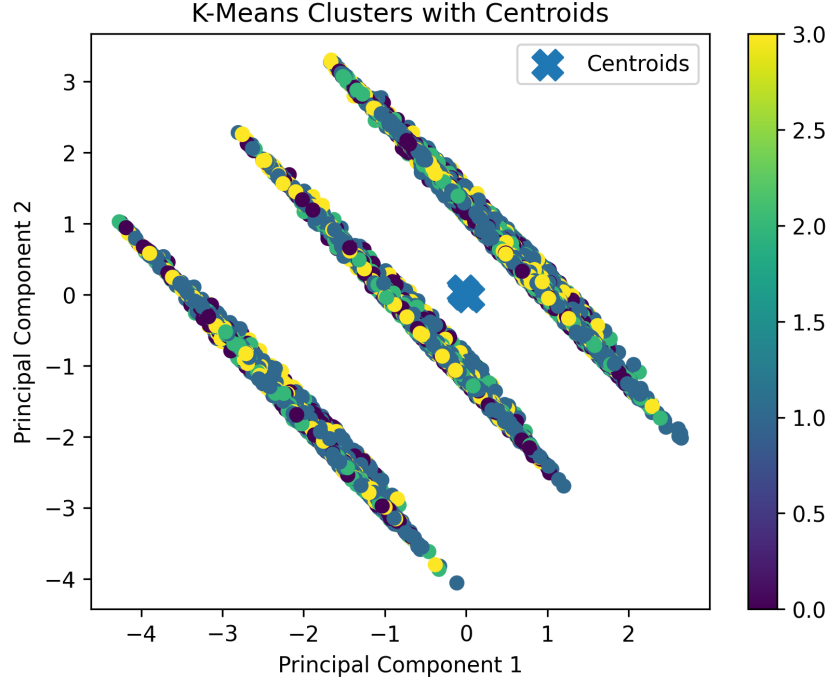


Figure 4: K-Means Clustering Performance

5.6.4 Final Analysis

Final Analysis of K-Means is that K-Means did not perform well for this dataset. The reason is that the data is not clearly separable into distinct clusters, and the features do not form natural groupings.

6 Model Selection & Comparison Analysis

In this section, the performance of all trained classification models is compared using multiple evaluation metrics including accuracy, precision, recall, F1-score, and confusion matrices. Visualizations such as bar charts are also used to provide a clear comparison.

6.1 Accuracy Comparison

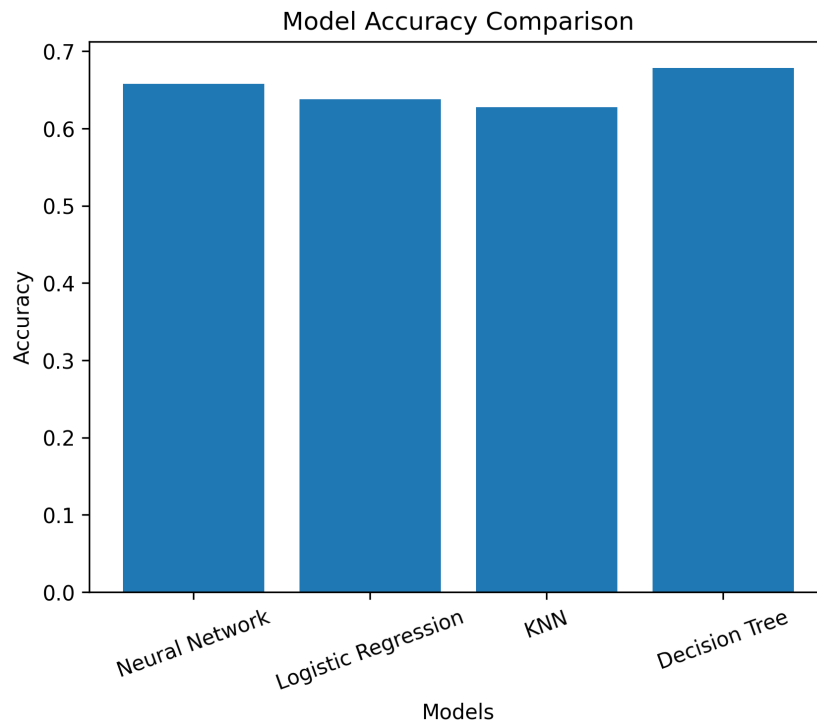


Figure 5: Model Accuracy Comparison

Accuracy Results

Model	Accuracy
Decision Tree	0.6782
Neural Network	0.6577
Logistic Regression	0.6377
KNN	0.6273

Table 1: Accuracy Comparison of Models

Analysis

Decision Tree achieved the highest accuracy

Neural Network performed second best

KNN showed the lowest accuracy

6.2 Precision and Recall Comparison

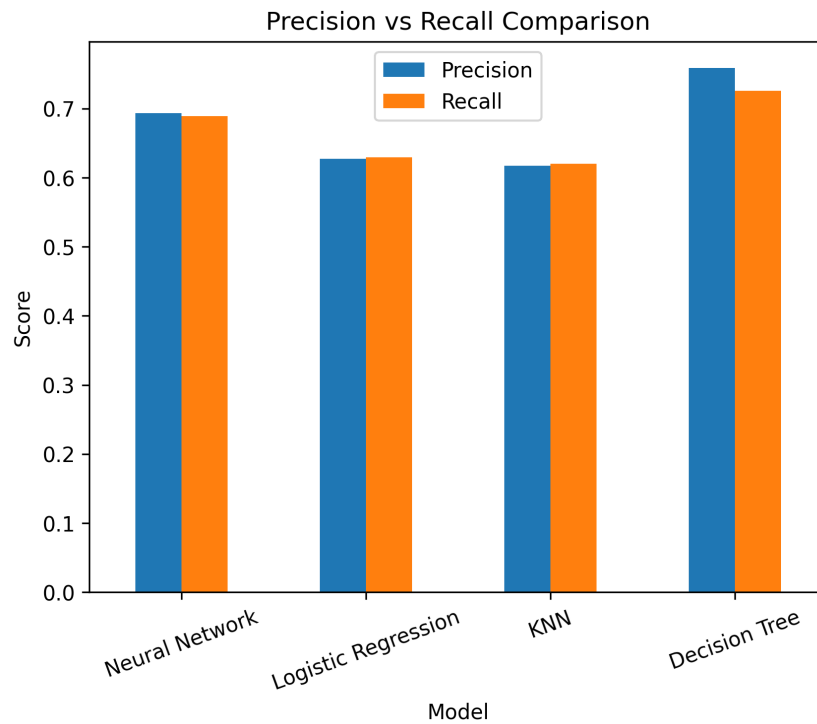


Figure 6: Precision vs Recall Comparison

Metric Summary

Model	Precision	Recall	F1 Score
Decision Tree	0.7586	0.7255	0.6747
Neural Network	0.6935	0.6887	0.6575
Logistic Regression	0.6270	0.6296	0.6277
KNN	0.6172	0.6199	0.6177

Table 2: Precision, Recall and F1 Score Comparison

Analysis

Decision Tree has the highest precision and recall, indicating strong classification ability

Neural Network provides balanced performance

Logistic Regression and KNN perform similarly but lower

6.3 Confusion Matrix Analysis

Confusion matrices provide a detailed breakdown of predictions for each model.

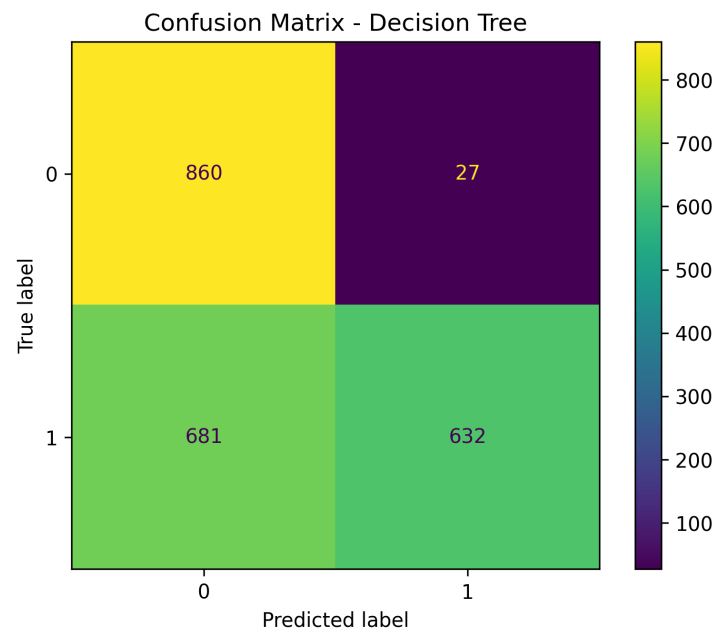


Figure 7: Decision Tree Confusion Matrix

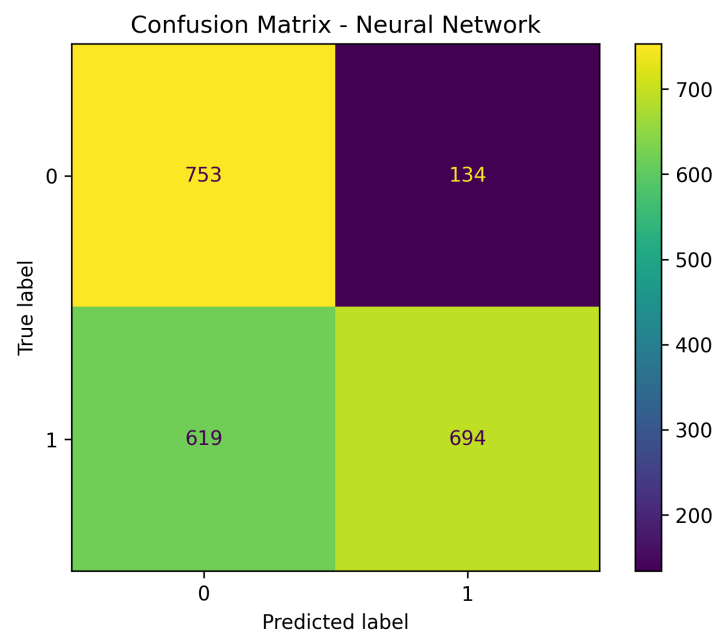


Figure 8: Neural Network Confusion Matrix

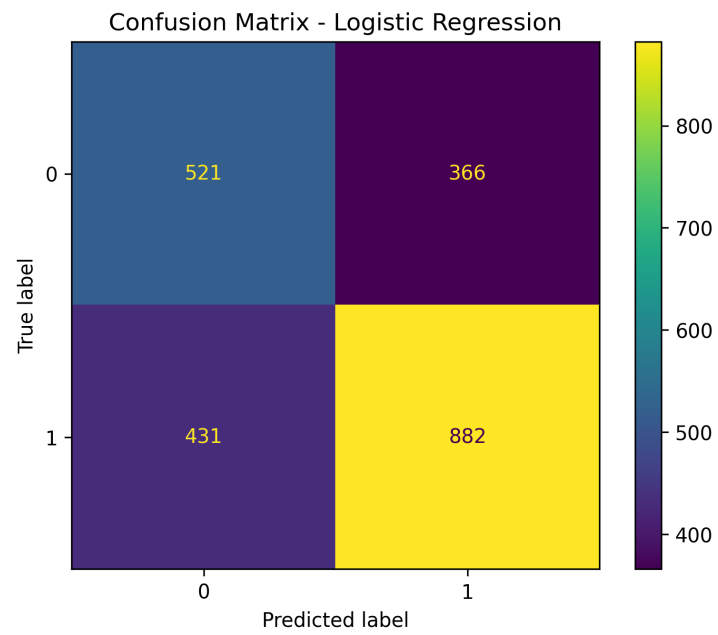


Figure 9: Logistic Regression Confusion Matrix

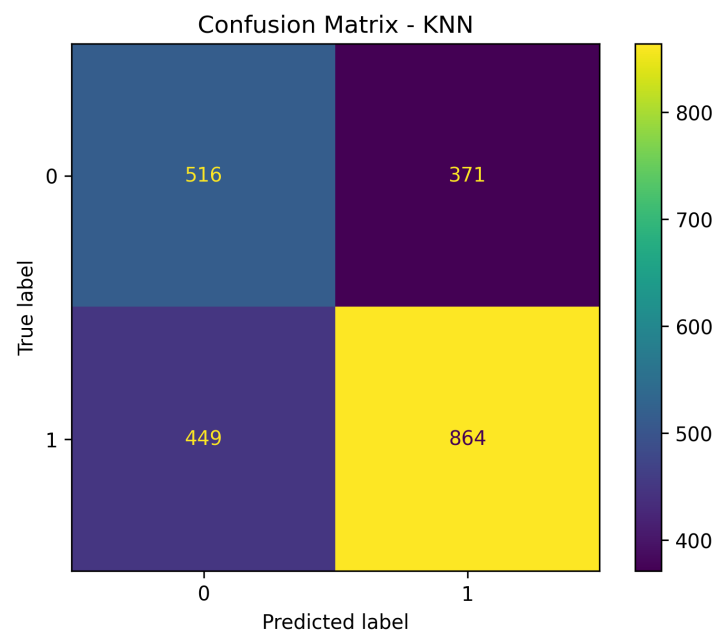


Figure 10: KNN Confusion Matrix

6.4 ROC Curve Analysis

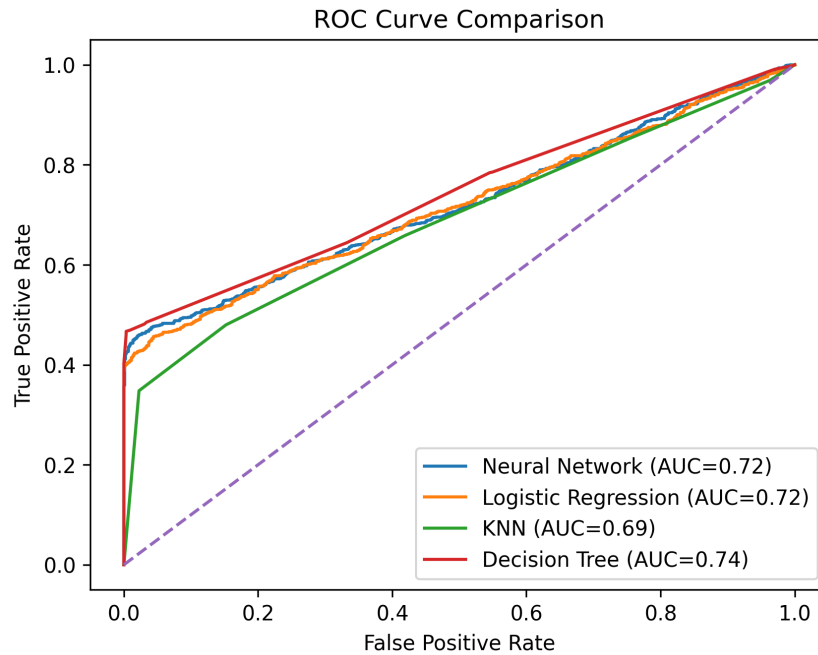


Figure 11: ROC Curve Comparison

The area under the curve (AUC) indicates the model's ability to distinguish between classes. Here is the optimal result came from Decision Tree and lowest is KNN

7 Conclusion

The results indicate that the Decision Tree model performed best with 67.82% accuracy, followed by Neural Network (65.77%), Logistic Regression (63.77%), and KNN (62.73%), suggesting that shipment delay prediction involves moderately complex, non-linear relationships but lacks strong predictive signals. K-Means clustering failed to identify meaningful patterns, confirming the absence of natural groupings in the dataset. These results are mainly due to weak feature-target correlations and moderate class imbalance (60/40 split).